Computer Networks Lab 1 Report

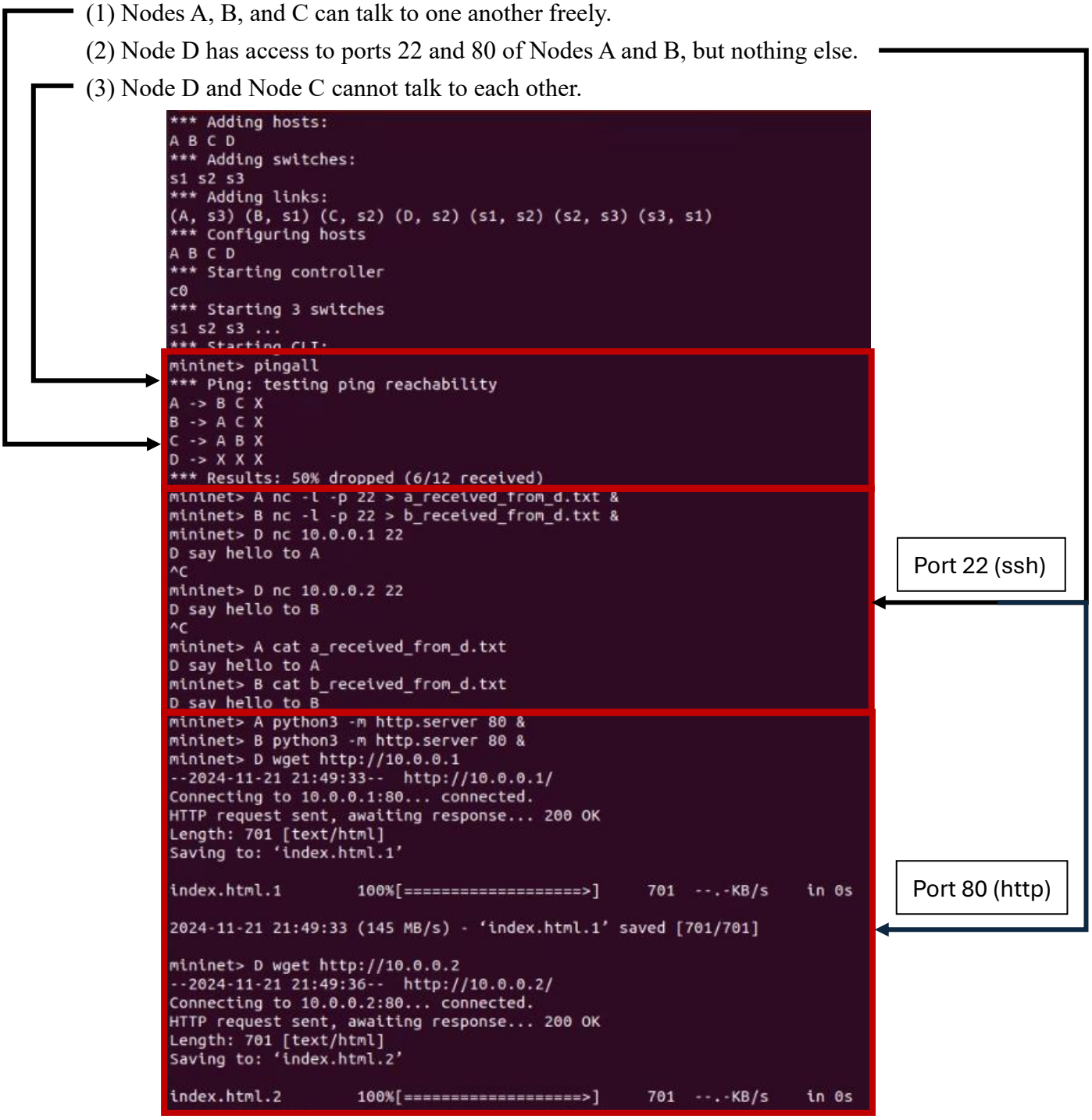
313551124 陳禹辰

Topic: SDN controller & Mininet

(1) Nodes A, B, and C can talk to one another freely.

(2) Node D has access to ports 22 and 80 of Nodes A and B, but nothing else.

(3) Node D and Node C cannot talk to each other.



上圖為實作結果，包含

(1) 實作要求的 topology

(2) pingall 的測試

(3) Node D 可以連結 Node A, B 的 port 22(ssh) 以及 port 80(http)

下面則是我的實作過程

1. 實作要求的 topology

透過設定檔案並且以 mininet 來建立

sudo mn --custom=sample.py --topo=mytopo --controller=remote,ip=127.0.0.1 --switch=ovsk

```
s1 = self.addSwitch('s1')
s2 = self.addSwitch('s2')
s3 = self.addSwitch('s3')

host_a = self.addHost('A', ip='10.0.0.1/24', mac='00:00:00:00:00:01')
host_b = self.addHost('B', ip='10.0.0.2/24', mac='00:00:00:00:00:02')
host_c = self.addHost('C', ip='10.0.0.3/24', mac='00:00:00:00:00:03')
host_d = self.addHost('D', ip='10.0.0.4/24', mac='00:00:00:00:00:04')

self.addLink(host_a, s3, port1=0, port2=1)
self.addLink(host_b, s1, port1=0, port2=3)  # B 的接口 eth0 -> S1 的端口 1
self.addLink(host_c, s2, port1=0, port2=3)  # C 的接口 eth0 -> S2 的端口 1
self.addLink(host_d, s2, port1=0, port2=2)  # D 的接口 eth0 -> S2 的端口 2

# 连接交换机之间的端口
self.addLink(s1, s2, port1=2, port2=4)  # S1 的端口 2 -> S2 的端口 3
self.addLink(s2, s3, port1=1, port2=2)  # S2 的端口 4 -> S3 的端口 2
self.addLink(s3, s1, port1=3, port2=1)  # S3 的端口 3 -> S1 的端口 3
```

```
cn@cn-VirtualBox:~/lab1$ sudo mn --custom=sample.py --topo=mytopo --controller=remote,ip=127.0.0.1 --switch=ovsk
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Unable to contact the remote controller at 127.0.0.1:6633
Setting remote controller to 127.0.0.1:6653
*** Adding hosts:
A B C D
*** Adding switches:
s1 s2 s3
*** Adding links:
(A, s3) (B, s1) (C, s2) (D, s2) (s1, s2) (s2, s3) (s3, s1)
*** Configuring hosts
A B C D
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3 ...
```

2. pingall 的測試

因為建立起來的 topology 為一個環形拓譜，如果只是使用一般的 ryu-manager ryu.app.simple_switch_13，很容易造成 broadcast storm 的情況，所以要先針對這部分做出處理。我有嘗試過 proxy arp 的處理方式，但是雖然可以讓 arp 的部分順利運行，不過卻會在後續 pingall 失效。所以後來採取類似 Spanning Tree Protocol (STP)相似的策略，透過設置 flow rule，讓 s1 以及 s2 之間的 link 不會有封包傳送，來避免 broadcast storm 的問題，讓 pingall 可以順利運行。

在 A, B, C 已經可以互相 ping 之後，還要針對 D 做出處理。對於 D 跟 C 之間，可以直接設置 flow rule 讓他們不處理來自彼此的封包

```
def deny_d_to_c(self, datapath, parser, ofproto):
    # Deny D (10.0.0.4) from communicating with C (10.0.0.3)
    match = parser.OFPMatch(eth_type=0x0800, ipv4_src='10.0.0.4', ipv4_dst='10.0.0.3')
    self.add_flow(datapath, 4, match, [])

    match = parser.OFPMatch(eth_type=0x0800, ipv4_src='10.0.0.3', ipv4_dst='10.0.0.4')
    self.add_flow(datapath, 4, match, [])
```

3. Node D 可以連結 Node A, B 的 port 22(ssh) 以及 port 80(http)

對於 A, B，我的作法是把 D 與 A, B 不處理封包的 flow rule priority 設置的比較低，這樣就不會影響到 port 22 以及 80 的 flow rule

```
def deny_d_to_ab_other_ports(self, datapath, parser, ofproto):
    # Deny D (10.0.0.4) from accessing any other ports on A (10.0.0.1) and B (10.0.0.2)
    allowed_ips = ['10.0.0.1', '10.0.0.2']
    for ip_dst in allowed_ips:
        match = parser.OFPMatch(eth_type=0x0800, ipv4_src='10.0.0.4', ipv4_dst=ip_dst, ip_proto=6)
        self.add_flow(datapath, 15, match, [])  # Add a drop rule with lower priority than the allowed ports
```

```
def allow_d_to_ab_ports(self, datapath, parser, ofproto):
    # Allow D (10.0.0.4) to access A (10.0.0.1) and B (10.0.0.2) on ports 22 and 80
    allowed_ips = ['10.0.0.1', '10.0.0.2']
    for ip_dst in allowed_ips:
        for port in [22, 80]:
            match = parser.OFPMatch(eth_type=0x0800, ipv4_src='10.0.0.4', ipv4_dst=ip_dst, ip_proto=6, tcp_dst=port)
            actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)]
            self.add_flow(datapath, 20, match, actions)  # Increase priority to ensure this rule is matched
```

並且也要設置 Node A, B 能夠針對 ssh 及 http 的做出回應

```
def allow_ab_to_d_response(self, datapath, parser, ofproto):
    # Allow A and B to respond to D (e.g., TCP SYN-ACK packets)
    allowed_ips = ['10.0.0.1', '10.0.0.2']
    for ip_src in allowed_ips:
        match = parser.OFPMatch(eth_type=0x0800, ipv4_src=ip_src, ipv4_dst='10.0.0.4', ip_proto=6)
        actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)]
        self.add_flow(datapath, 18, match, actions)  # Slightly lower priority than allow_d_to_ab_ports
```